



CSE103: Structured Programming (Section-19)
Summer 2026

Project Report

Personal Expense Tracker
<https://github.com/abkrohan/Personal-Expense-Tracker/>

Submitted by:

Student ID	Student Name	Contribution %
2026-2-60-014	Mohammad Abdullah Bin Khan	35%
2026-2-60-034	Protik Paul	30%
2026-2-50-038	Merajul Islam Provat	25%
2026-2-50-013	Md. Jubayer Hossain	10%

1. Project Description

Personal Expense Tracker is a C-language console application that helps a student record and monitor daily spending. In everyday life, students often lose track of where their money goes because expenses are scattered across memory, paper notes, or nothing at all. This program solves that problem by letting the user log every expense under a category (such as Food, Transport, or Shopping), view all recorded expenses at any time, search expenses by category, update a wrong entry, and see the total amount spent so far.

The real-world scenario this project targets is a common one for university students living on a tight, irregular budget: a typical day might involve a small breakfast purchase, a bus or rickshaw fare, lunch at the cafeteria, and an impulse purchase from a nearby shop. None of these individually feels significant, but over a week or a month they add up in ways that are hard to reconstruct after the fact. Because the amounts are small and the number of transactions is high, pen-and-paper tracking is quickly abandoned, and spreadsheet tools are often seen as too heavyweight for a quick entry between classes. A lightweight console tool that takes less than ten seconds to log an expense fits naturally into that gap.

Objective: To apply structured programming concepts (arrays, structures, functions, loops, and file handling) to build a small but complete record-management system. Specifically, the project set out to:

- Represent a real-world entity (an expense) as a C structure with meaningful fields.
- Manage a collection of these records using an array, keeping the code simple enough for a first structured-programming project.
- Split every piece of behaviour into its own function, so that `main()` reads like a table of contents for the whole program.
- Validate every piece of user input at the point where it is entered, rather than trusting it and dealing with bad data later.
- Persist data across runs using simple text-based file I/O, so the tool is actually useful from one session to the next.

Expected Input: Menu choice (1-6), category name (up to 19 characters), and expense amount (a non-negative number), entered through the console one prompt at a time.

Expected Output: A formatted table of expenses, category-filtered search results, an update confirmation message, and the total amount spent so far, along with a persistent `expenses.txt` file that stores all data between runs. Error conditions — invalid menu choices, non-numeric or negative amounts, a full expense list, a missing or unreadable file — all produce a clear, specific message rather than a crash or silent failure.

Scope and Assumptions: the project intentionally targets a single user working with a single data file on one machine — there is no login system, no multi-user separation, and no networked storage. Categories are free-text rather than a fixed list, so the user can introduce a new category simply by typing it; this keeps the program simple but does mean two categories that differ only in spelling or capitalisation ("Food" vs "food") are treated as different categories by `searchCategory()`, since `strcmp()` is case-sensitive. These are reasonable simplifications for a structured-programming assignment whose purpose is to demonstrate correct use of arrays, structures, functions, and file I/O rather than to ship a production-ready finance application.

2. Key Features

The application is built around six user-facing features, each implemented as its own function, plus a set of shared validation and persistence helpers that every feature relies on.

Add Expense

Lets the user log a new expense under a category with an amount. The record's ID is generated automatically as count + 1, so the user never has to think about ID management. The category is read with a 19-character limit to prevent buffer overflow, and the amount is read through readValidAmount(), which rejects anything non-numeric or negative before it ever reaches the array.

Display All

Prints every recorded expense in a clean, column-aligned table showing ID, Category, and Amount. If no expenses have been added yet, the function prints a friendly "No expenses yet" message instead of an empty table, so the user is never left staring at a blank screen wondering if something went wrong.

Search by Category

Filters the full list down to only the records whose category matches the user's input exactly, using strcmp() for comparison. All matching records are printed in the same table format as Display All. If nothing matches, the function reports "No expense found in this category" rather than showing nothing and leaving the user unsure whether the search actually ran.

Update Expense

Allows the user to correct a category or amount by referencing the expense's ID. The function scans the array for a matching ID; if found, it re-prompts for a new category and a new (validated) amount and overwrites the existing record in place. If the ID does not exist, it reports "ID not found" without modifying anything.

Show Total

Loops through every record, sums the amount field, and reports both the total number of expenses and the total amount spent, formatted to two decimal places so currency values always look correct regardless of how the individual amounts were entered.

Save & Load

Gives the program memory across sessions. On exit (menu choice 6), every record is written to expenses.txt as "id category amount" on its own line. On startup, loadFile() reads that file back in, one line at a time, stopping cleanly at the array limit or end of file. A missing file on first run is treated as a normal, expected case rather than an error — the program simply starts with an empty list.

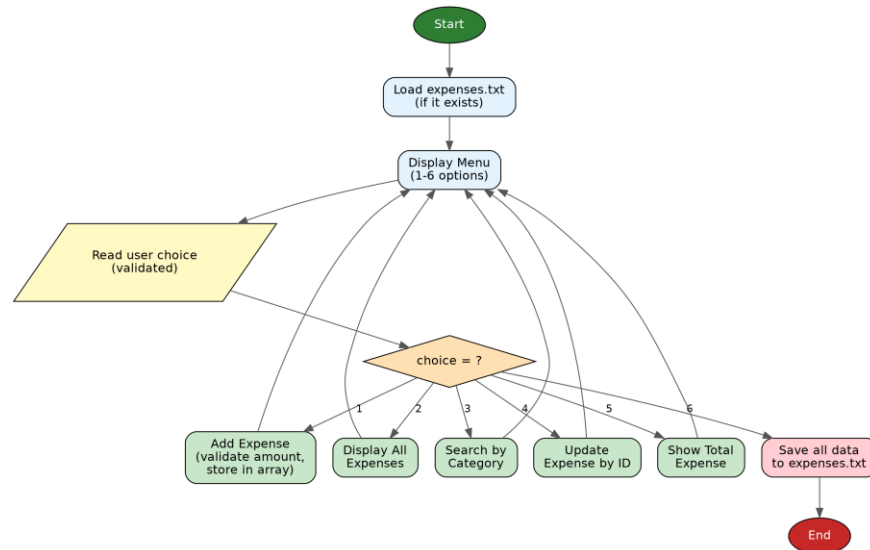
The table below maps each feature to the function that implements it and the validation applied before any data is changed, as a quick-reference summary of the previous six sub-sections.

Feature	Function	Validation Applied
Add Expense	addExpense()	Array-full check, 19-char category limit, non-negative amount
Display All	displayExpense()	Empty-list check before printing the table
Search by Category	searchCategory()	Empty-list check; "not found" message on no match
Update Expense	updateExpense()	ID-exists check; same amount validation as Add
Show Total	totalExpense()	None needed — reads existing validated data only
Save & Load	saveFile() / loadFile()	NULL check on fopen(); partial-record protection on load

3. Program Design

3.1 Flowchart

The flowchart below shows the overall control flow of the program, from loading saved data at startup to looping through the menu until the user chooses to save and exit.



Reading the diagram top to bottom: the program starts by attempting to load any previously saved data from `expenses.txt` — this step succeeds silently whether or not the file exists, since a missing file simply means this is the first run. The program then enters its main loop, which displays the six-option menu and reads a validated choice from the user. Based on that choice, control branches to exactly one of the six feature functions (or, for an invalid choice, back to the menu with an error message). Every branch except option 6 returns control to the menu, so the user can perform any number of operations in a single session. Choosing option 6 breaks out of the loop, writes the current state of the array to disk, and the program terminates.

3.2 Program Structure

The program is organized around one structure and eleven functions, all operating on a single global array:

```
#define MAX 100
#define FILENAME "expenses.txt"

struct Expense {
    int id;
    char category[20];
    float amount;
};

struct Expense list[MAX]; // array of all expense records
int count = 0;           // number of records currently stored
```

MAX caps the program at 100 stored expenses, which keeps the data structure simple, easy to manage and is more than enough for a semester-length demonstration project. FILENAME centralises the name of the data file in one place, so it only has to be changed once if the storage location ever needs to move.

main() — loads saved data, displays the menu in a loop, and dispatches to the right function based on user choice. The loop is a do-while, guaranteeing the menu is shown at least once even before any input has been read.

addExpense() — creates a new record with an auto-generated ID, reads a validated category and amount, and appends it to the array. Before doing anything else it checks `count >= MAX` and refuses to add a new record once the array is full, printing "Expense list is full!" instead of writing past the end of the array.

displayExpense() — prints every record in the array as a formatted table, using `printf` column widths so that ID, Category, and Amount line up regardless of how long each category name is.

searchCategory() — scans the array and prints only the records matching a given category, using a simple linear search since the array is small enough that a more advanced search structure would add complexity without a meaningful performance benefit.

updateExpense() — finds a record by ID and overwrites its category and amount, using the same validated-input helpers as `addExpense()` so an update can never introduce a negative or non-numeric amount.

totalExpense() — sums the amount field across the array and prints the total, along with a count of how many expenses were included in the sum.

saveFile() / loadFile() — write the array to `expenses.txt` and read it back, using `fopen()`'s return value to detect and report file errors rather than assuming the file system operation always succeeds.

readValidInt() / readValidAmount() / clearInputBuffer() — shared helper functions that keep re-prompting until the user provides valid input, and clear leftover characters from the input buffer after an invalid entry so a single bad keystroke cannot desynchronise every prompt that follows it.

3.3 Design Rationale and Limitations

Grouping id, category, and amount into a single struct, and keeping every record in one flat array, was a deliberate choice to keep the data model simple enough to reason about at this stage of learning C. It also means the same basic pattern — loop through `list[0..count-1]` — is reused across Display, Search, Update, and Total, instead of each feature needing its own data-access logic.

This design does have known limitations, which the team is aware of rather than having overlooked: the fixed-size array means the program cannot grow past `MAX` expenses without recompiling with a larger constant or moving to dynamic memory allocation; search and update both run in linear time, which is invisible at this scale but would not stay fast for a much larger dataset; and the plain-text file format is easy to inspect and debug but offers no protection against a user manually editing `expenses.txt` into an invalid state. These trade-offs were judged acceptable for a structured-programming exercise whose primary goal is demonstrating arrays, structures, functions, and file I/O working together correctly.

3.4 Testing and Sample Test Cases

Each function was tested individually with a small set of hand-picked inputs chosen to cover the normal case, a boundary case, and at least one invalid case, before being wired into the shared menu. The table below summarises the core test cases run against the final build.

#	Function	Input	Expected Output	Result
1	<code>addExpense()</code>	<code>category="Food", amount=150</code>	"Expense added successfully!", ID=1	Pass
2	<code>addExpense()</code>	<code>amount=-20</code>	"Amount cannot be negative! Enter again:"	Pass
3	<code>addExpense()</code>	<code>amount="abc"</code>	"Invalid amount! Please enter a number:"	Pass
4	<code>addExpense()</code>	<code>count already = MAX</code>	"Expense list is full!"	Pass
5	<code>displayExpense()</code>	<code>count = 0</code>	"No expenses yet."	Pass

#	Function	Input	Expected Output	Result
6	searchCategory()	category="Transport" (exists)	Matching row(s) printed	Pass
7	searchCategory()	category="Rent" (absent)	"No expense found in this category."	Pass
8	updateExpense()	id=1 (exists)	"Expense updated successfully!"	Pass
9	updateExpense()	id=99 (absent)	"ID not found."	Pass
10	loadFile()	expenses.txt missing	Starts with an empty list, no error shown	Pass
11	saveFile()	no write permission on folder	"Error! Could not save data."	Pass
12	main() menu	choice="9"	"Invalid choice! Please choose 1-6."	Pass

All twelve cases passed on the final build. The boundary case (test 4, a full array) and the two "not found" cases (7 and 9) were the ones most likely to be missed during early development, since they only surface when the program is pushed slightly outside its everyday usage pattern — which is exactly why they were kept in the final test pass rather than dropped once the happy-path cases were working.

3.5 Glossary of Key Terms

A short glossary of the structured-programming terms used throughout this report, for reference:

- **Structure (struct):** a user-defined type that groups several related variables — here, id, category, and amount — under one name, so a single Expense can be passed around and stored as one unit instead of three separate variables.
- **Array:** a fixed-size, indexed collection of values of the same type. list[MAX] holds up to 100 struct Expense values, and count tracks how many of those slots are actually in use.
- **Function:** a named, reusable block of code that performs one task. Splitting the program into functions such as addExpense() and totalExpense() is what "structured programming" refers to in the assignment title — each function has one clear job and can be tested on its own.
- **Modularity:** the practice of keeping main() free of implementation detail — it only shows the menu and calls the matching function — so the overall program structure stays readable even as individual functions grow more complex.
- **File I/O:** reading from and writing to a file on disk. fopen() opens expenses.txt, fprintf()/fscanf() write and read individual records, and fclose() releases the file when done.
- **Input validation:** checking that user input meets the program's expectations (a real number, a non-negative amount, a reasonable string length) before using it, instead of trusting it and risking a crash or corrupted data.
- **Input buffer:** the temporary storage scanf() reads from. When scanf() fails to match the expected format, leftover characters remain in the buffer; clearInputBuffer() discards them so the next prompt starts clean.

4. Tools & Technologies Used

- **Language:** C (C99) — chosen because it is the language taught in CSE103 and maps directly onto the course's structured-programming concepts.
- **Compiler:** GCC — compiles the program from the command line with a single gcc expense_tracker.c -o expense_tracker call and gives clear warnings for common mistakes such as format-specifier mismatches.
- **Editor/IDE:** Code::Blocks — used for writing, running, and stepping through the program during debugging.

- **Operating System:** Windows 11 Pro — the program uses only standard C library calls, so it compiles and runs unchanged on either platform.
- **Standard Libraries:** `stdio.h` for all console and file input/output (`printf`, `scanf`, `fopen`, `fprintf`, `fscanf`), and `string.h` for category comparison via `strcmp()`.

No third-party or platform-specific libraries were used. This was a deliberate constraint: relying only on the C standard library keeps the project fully portable and keeps the focus on the structured-programming concepts the course is assessing, rather than on library integration.

5. Role Assignment

The work was split so that each member owned a coherent slice of the program rather than a scattering of unrelated lines, which made it easier for each person to test their own part in isolation before integrating everything together.

- **Mohammad Abdullah Bin Khan — Core logic:** `addExpense()`, `updateExpense()`, `totalExpense()`. Responsible for the functions that create and modify records, including deciding how ID generation and record overwriting would work.
- **Protik Paul — Input/output and validation:** `readValidInt()`, `readValidAmount()`, `displayExpense()`, `searchCategory()`. Responsible for making sure no invalid input could ever reach the data layer, and that output was always readable.
- **Merajul Islam Provat — File handling and testing:** `saveFile()`, `loadFile()`, overall testing & debugging, report writing. Responsible for persistence, plus running the full test pass described in Section 6 before the final submission.
- **Md. Jubayer Hossain — Documentation and Presentation:** Overall project coordination, main program structure, integration

Integration happened incrementally: each function was written and tested against a small hand-typed dataset before being wired into the shared menu in `main()`, which kept merge conflicts and last-minute surprises to a minimum.

6. Sample Input/Output with Screenshots

```

===== EXPENSE TRACKER =====
1. Add Expense
2. Display All Expenses
3. Search by Category
4. Update Expense
5. Show Total Expense
6. Save & Exit
Enter choice: |

===== EXPENSE TRACKER =====
1. Add Expense
2. Display All Expenses
3. Search by Category
4. Update Expense
5. Show Total Expense
6. Save & Exit
Enter choice: 2

ID   Category   Amount
-----
1    Food       150.00
2    Transport  60.00
3    Shopping   300.00

===== EXPENSE TRACKER =====
1. Add Expense
2. Display All Expenses
3. Search by Category
4. Update Expense
5. Show Total Expense
6. Save & Exit
Enter choice: 1

--- Add Expense ---
Enter category (e.g. Food): Food
Enter amount: 150
Expense added successfully!

```

To make the expected behaviour clear even before the screenshots are inserted, a representative console session is reproduced below. This is the exact text the program prints for a short session: opening the menu, adding two expenses, displaying them, and requesting the total.

```

===== EXPENSE TRACKER =====
1. Add Expense
2. Display All Expenses
3. Search by Category
4. Update Expense
5. Show Total Expense

```

```
6. Save & Exit
Enter choice: 1
Enter category (e.g. Food): Food
Enter amount: 150
Expense added successfully!

Enter choice: 1
Enter category (e.g. Food): Transport
Enter amount: 60
Expense added successfully!

Enter choice: 2
ID      Category      Amount
1       Food           150.00
2       Transport      60.00

Enter choice: 5
Total expenses: 2
Total amount: 210.00
```

A second short session illustrates how invalid input is handled without crashing the program:

```
Enter choice: 9
Invalid choice! Please choose 1-6.
Enter choice: abc
Invalid input! Please enter a number: 1
Enter category (e.g. Food): Food
Enter amount: -50
Amount cannot be negative! Enter again: 50
Expense added successfully!
```

A third session shows searching by category and then updating an existing record, which together exercise the two functions most likely to encounter a "not found" case:

```
Enter choice: 3
Enter category: Food
ID: 1
Category: Food
Amount: 150.00

Enter choice: 4
Enter ID to update: 2
Enter new category: Bus Fare
Enter new amount: 45
Expense updated successfully!

Enter choice: 4
Enter ID to update: 99
ID not found.
```

These three sessions together cover every menu option at least once, plus the two most common error paths (invalid menu choice and invalid amount) and the two most common "not found" paths (search and update), giving reasonable confidence that the program behaves correctly across both normal and edge-case usage.

7. Conclusion

Through this project we learned how to combine arrays, structures, functions, and file handling into one working C program, and why breaking a program into small, single-purpose functions makes it far easier to write, test, and explain. Writing the input-validation helper functions also showed us how easily a program can crash or loop forever on bad input if `scanf()`'s return value is ignored, and why clearing the input buffer after a failed `scanf()` matters — a single stray character left behind by an invalid `scanf()` call is enough to desynchronise every prompt that follows it.

One difficulty we faced was an early version of the program silently letting invalid or negative amounts into the data — this taught us to validate at the point of input rather than trying to filter bad data out later. We also ran into a handful of smaller issues typical of a first larger C program: off-by-one mistakes when checking count against `MAX`, forgetting to `fclose()` a file handle on an early return path, and format-string mismatches between `scanf()` and the struct fields being read into. Each of these was found through deliberate, small-scale testing — running one function at a time with a handful of hand-picked inputs — rather than by trying to test the whole program at once.

Working as a team also reinforced a more process-oriented lesson: agreeing on the struct layout and function signatures before writing any implementation code meant that each member's part could be integrated with almost no rework, whereas a mismatched field name or parameter order discovered late would have cost far more time to fix than the few minutes spent agreeing on it up front.

Future improvements could include:

- A delete-expense feature, so incorrect entries can be removed rather than only corrected.
- Monthly or category-wise summaries, to make longer-term spending patterns visible at a glance.
- Sorting expenses by amount or date, to make the display table easier to scan.
- Recording the date and time of each expense, which would enable the summaries above.
- A basic budgeting feature that warns the user when spending in a category passes a set limit.
- Replacing the plain-text file format with a lightweight structured format, to make the saved data more robust against manual editing.

8. References

- Balagurusamy, E. Programming in ANSI C.
- GCC documentation — <https://gcc.gnu.org/onlinedocs/>
- W3Schools C Tutorial — <https://www.w3schools.com/c/>
- GeeksforGeeks C Programming — <https://www.geeksforgeeks.org/c-programming-language/>
- Class lecture notes, CSE103 — Structured Programming